Software Architecture and Design Document

Supply Chain Management Simulator

Software Engineering Mini Project

Document ID	SCM-SAD-001
Project	Supply Chain Management Simulator
Version	1.0
Date	11-10-2026
Status	Draft
Authors	Siri Krishna C R, Spoorthi Poornachandra, D R Rushmitha, Kallepalli Sai Meghana, Shraddha
Related documents	Software Requirements Specification v1.0 (SCM-SRS-001), Software Test Plan v1.0

Revision History

Version	Date	Author	Change Summary
0.1	10-10-2026	Project team	Initial architecture outline, module decomposition and ownership
1.0	11-10-2026	Project team	Complete architecture and design: UML views, decisions, interfaces, threat model and traceability

Approvals

Role	Name	Signature / Date
Course Coordinator		
Repository owner and scrum master	Kallepalli Sai Meghana	
QA lead	D R Rushmitha	
Developer	Siri Krishna C R	
Developer	Spoorthi Poornachandra	
Developer	Shraddha	

1. Introduction

1.1 Purpose

This document describes the software architecture and design of the Supply Chain Management Simulator (SCM Simulator). It records the architectural decisions taken, the static structure of the system, the behaviour of its key runtime flows, and the interfaces between its components. It is the reference for implementation, code review and testing, and it is traceable to the Software Requirements Specification (SRS) and the Software Test Plan (STP).

1.2 Scope

The SCM Simulator is a command-line application that simulates, one day at a time, a single warehouse that supplies multiple products to customers and replenishes its stock from multiple suppliers. The architecture covers the following capabilities:

- Customer demand generation using constant, uniform random and seasonal patterns.
- Customer order fulfilment on a first-in-first-out basis, with unmet demand held as backorders.
- Demand forecasting by moving average or exponential smoothing, with forecast error measurement.
- Replenishment decisions using safety stock, reorder point and Economic Order Quantity (EOQ).
- Supplier selection by price, lead time or a weighted balance of price, lead time and reliability.
- Purchase order lifecycle management, including reliability-based delivery delays.

- Inbound transport costing, cost accounting, KPI computation and reporting to CSV and console.

Out of scope: multi-warehouse or multi-echelon networks, integration with real ERP or supplier systems, a graphical user interface, persistent databases, and user accounts or authentication.

1.3 Audience

- **Developers** implementing and reviewing the modules described in Sections 3 and 4.
- **Test engineers and reviewers** deriving and verifying test cases against the design.
- **Course faculty and evaluators** assessing the architecture, process compliance and traceability.
- **Future maintainers** extending the simulator, for example with additional warehouses or policies.

1.4 Definitions

Table 1: Acronyms and abbreviations

Term	Definition
ADR	Architecture Decision Record
CI	Continuous Integration
CLI	Command-Line Interface
COGS	Cost of Goods Sold
CSV	Comma-Separated Values
EOQ	Economic Order Quantity
FIFO	First In, First Out
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
MAE / MAPE	Mean Absolute Error / Mean Absolute Percentage Error
PO	Purchase Order
RNG	Random Number Generator
ROP	Reorder Point
RTM	Requirements Traceability Matrix
SRS / STP	Software Requirements Specification / Software Test Plan
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
UML	Unified Modeling Language

2. Document Overview

2.1 How to use this document

Section 3 describes the architecture: goals and constraints, stakeholders, the logical and deployment views, component responsibilities, architecture decisions, technology choices, risks, traceability and security architecture. Section 4 describes the design: the processing

model, behavioural views (state and sequence diagrams), component interfaces, error handling and the user experience. Section 5 contains the glossary, references and tools.

The document follows the viewpoint approach of ISO/IEC/IEEE 42010. The views used are:

Table 2: Architecture views

View	Concern addressed	Location
Logical view	Decomposition into components and classes, and their dependencies	Section 3.3, 3.4
Deployment view	Runtime environment and delivery pipeline	Section 3.6
Behavioural view	Object lifecycles and the main runtime interactions	Section 4.1, 4.2
Interface view	Contracts between components and with the user	Section 4.3
Decision view	Significant architecture decisions and alternatives	Section 3.5
Security view	Assets, trust boundaries, threats and controls	Section 3.9

Requirement identifiers follow the SRS: SCM-F-nnn (functional), SCM-NF-nnn (non-functional) and SCM-SR-nnn (security). Test case identifiers follow the STP, for example TC-INV-07.

2.2 Related Documents

Table 3: Related documents

Document	Version	Relationship
Software Requirements Specification (SRS)	1.0	Source of all requirements realised by this design
Software Test Plan (STP)	1.0	Verifies the design through the test cases referenced in Section 3.8
Requirements Traceability Matrix (RTM)	1.0	Maps requirements to modules and tests; SRS Section 8
Team Context Document	1.0	Development workflow, module ownership and delivery plan
Source repository scm-simulator, including .github/workflows/ci.yml	main branch	Implementation of this design and its CI pipeline

3. Architecture

3.1 Goals & Constraints

Table 4: Architecture goals

ID	Goal	Driver
AG-1	Correctness: every functional rule is verified by automated tests that run on every change.	SCM-NF-003
AG-2	Reproducibility: the same configuration and seed always produce identical results.	SCM-NF-002
AG-3	Configurability: all scenario data (products, suppliers, demand, costs, policy) is supplied through configuration; none is hardcoded.	SRS 2.5
AG-4	Modifiability: components interact only through published interfaces, so four developers can build them in parallel.	SCM-NF-003

ID	Goal	Driver
AG-5	Performance: a 365-day run of the reference scenario completes in under 2 seconds on a standard laptop.	SCM-NF-001
AG-6	Portability: runs on Windows, macOS and Linux with Python 3.12 and no third-party runtime dependencies.	SCM-NF-005
AG-7	Secure delivery: no unreviewed or untested change can reach the main branch.	SCM-SR-004

Table 5: Architecture constraints

ID	Constraint
C-1	Python 3.12 using the standard library only at runtime; pytest is used for testing.
C-2	Command-line interface only; single process on a single machine; no network access at runtime.
C-3	One warehouse; time advances in fixed steps of one day.
C-4	Mandated development process: one shared GitHub repository, protected main branch, one feature branch per feature, pull request with at least one approval, and CI tests that must pass before merge.
C-5	Team of four with equal contribution; the system is partitioned so that each member owns distinct modules.

3.2 Stakeholders & Concerns

Table 6: Stakeholders and concerns

Stakeholder	Concerns	Addressed in
Supply Chain Analyst (end user)	Trustworthy, explainable results; ability to compare policies and scenarios	3.6, 4.1, 4.5
Scenario Planner	Change products, suppliers and demand without code changes; clear validation errors	3.6, 4.3.5, 4.4
Developers	Parallel work without conflicts; stable interfaces; fast feedback on defects	3.4, 3.5, 4.3
Test engineers and reviewers	Testability of every rule; traceability from requirement to test	3.8, 4.3
Course faculty and evaluators	Process compliance (CI, protected main, reviews), documentation quality, balanced contribution	3.6, 3.9
Maintainers	Low coupling, documented decisions, room to extend	3.5, 4.6

3.3 Component (UML) Diagram

Figure 1 shows the logical decomposition into four layers. Dependencies point downwards only: the Presentation layer uses the Application and Data Access layers, the Application layer uses the Domain layer, and Domain components do not depend on the layers above them. The Simulation Engine is the only component that orchestrates other domain components.

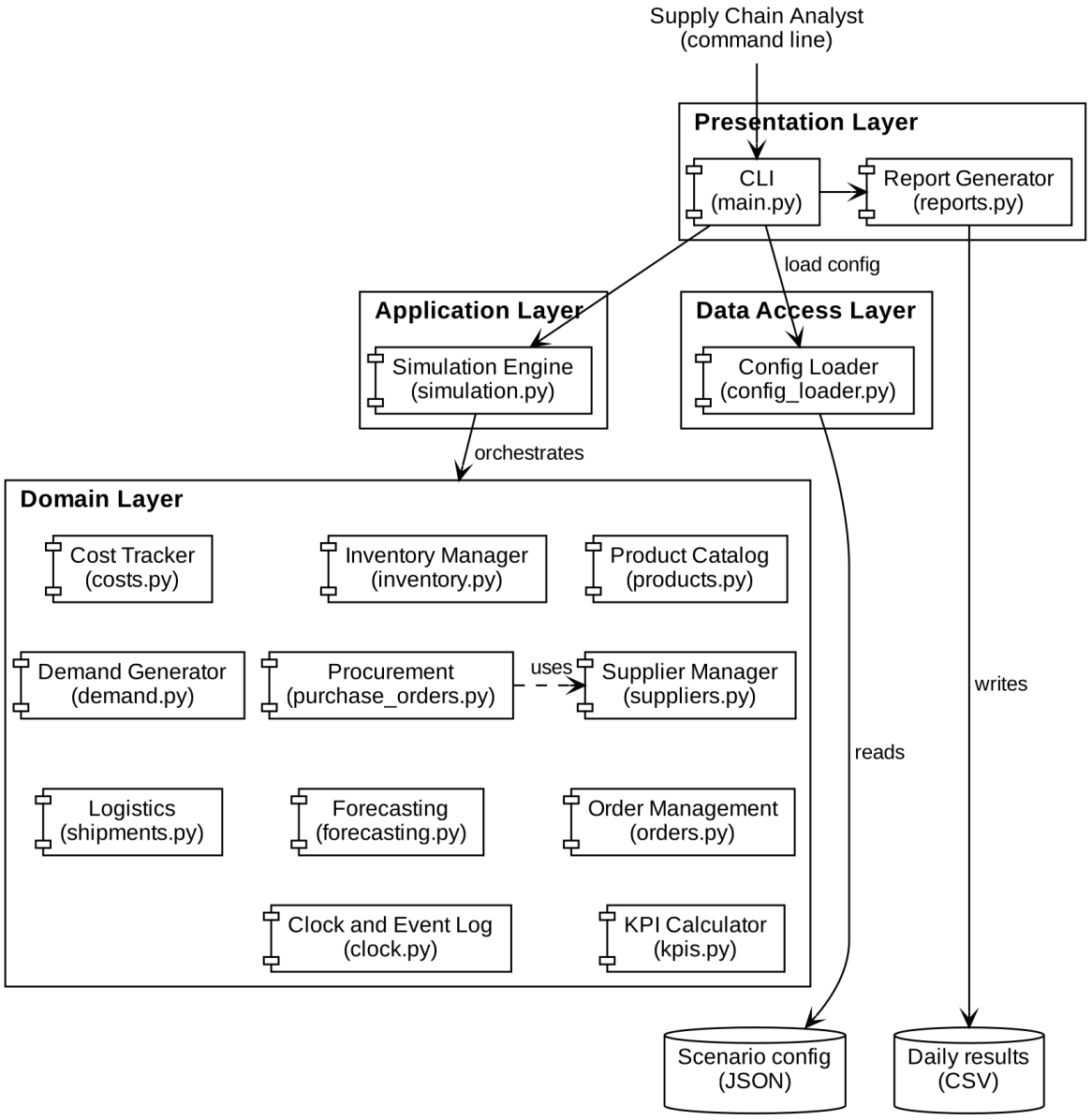


Figure 1: Component diagram (layered logical view)

Figure 2 shows the principal classes and their relationships. The SupplyChainSimulation class composes one instance of each domain service and one DemandGenerator per product. Catalogues and order books aggregate their entity objects.

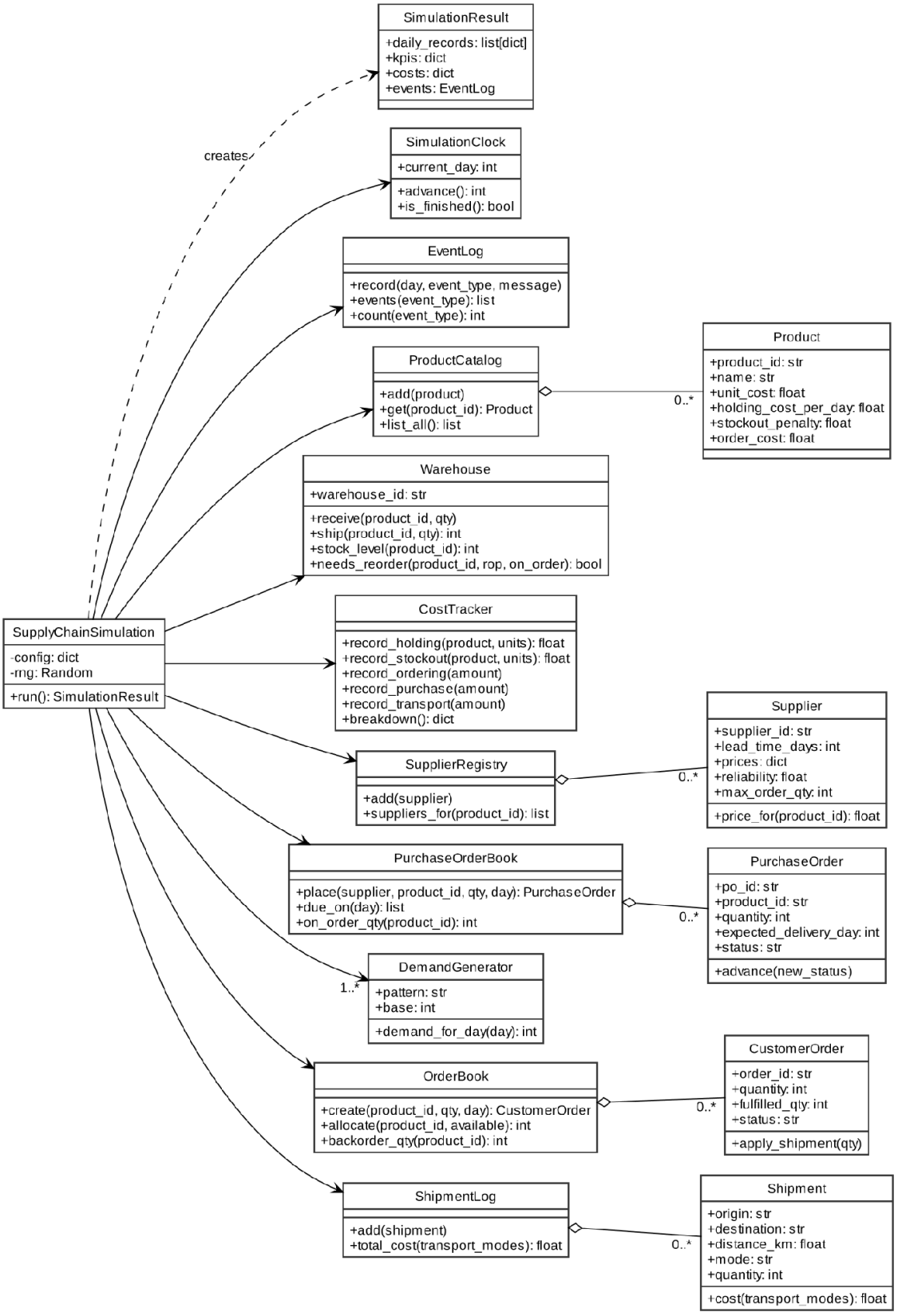


Figure 2: UML class diagram

3.4 Component Descriptions

Table 7: Component responsibilities and interfaces

Component (module)	Responsibility	Provided interface	Depends on
CLI (main.py)	Parses arguments, loads configuration, runs the simulation, exports results, prints the summary and maps errors to exit codes.	main(argv)	Config Loader, Simulation Engine, Report Generator
Report Generator (reports.py)	Writes daily records to CSV with formula-injection protection; formats the text summary.	export_daily_csv, format_summary	None
Simulation Engine (simulation.py)	Holds simulation state, executes the daily loop, orchestrates domain components, assembles the result.	SupplyChainSimulation. run, SimulationResult	All Domain components
Product Catalog (products.py)	Product master data and validation.	Product, ProductCatalog	None
Inventory Manager (inventory.py)	Stock levels per product; EOQ, safety stock and reorder point calculations; reorder decision.	Warehouse, economic_order_quantity, safety_stock, reorder_point	None
Cost Tracker (costs.py)	Accumulates holding, stockout, ordering, purchase and transport cost.	CostTracker	Product
Supplier Manager (suppliers.py)	Supplier master data, lookup by product, selection strategy.	Supplier, SupplierRegistry, select_supplier	None
Procurement (purchase_orders.py)	Purchase order creation, status lifecycle, open-order tracking, delivery delays.	PurchaseOrder, PurchaseOrderBook, apply_random_delay	Supplier
Demand Generator (demand.py)	Daily demand per configured pattern.	DemandGenerator	None
Order Management (orders.py)	Customer orders, FIFO allocation of stock, backorder tracking.	CustomerOrder, OrderBook	None
Forecasting (forecasting.py)	Moving average, exponential smoothing, demand variability, forecast error.	moving_average, demand_std, exponential_smoothing, mean_absolute_error, mean_absolute_percentage_error	None
Logistics (shipments.py)	Inbound shipments and mode-based transport cost.	Shipment, ShipmentLog	None
KPI Calculator (kpis.py)	Fill rate, service level, average inventory, inventory turnover.	compute_kpis and helper functions	None
Clock and Event Log (clock.py)	Simulation day counter and structured event recording.	SimulationClock, EventLog	None
Config Loader (config_loader.py)	Reads JSON, enforces size limit, validates schema, ranges and references.	load_config, validate_config	None

3.5 Chosen Architecture Pattern and Rationale

Chosen pattern: layered architecture with a central orchestrator. The system is organised into Presentation, Application, Domain and Data Access layers. The Simulation Engine in the Application layer is the single orchestrator; Domain components are cohesive, independently testable units that hold their own state and expose small interfaces.

Rationale:

- The workload is a deterministic batch computation with a feedback loop (today's reorder affects future stock), which maps naturally onto one orchestrator calling domain services in a fixed order.
- Isolating Domain components lets each be unit-tested without the engine and lets four developers work in parallel with no shared files.
- Keeping file and console I/O in the outer layers makes the Domain layer free of side effects other than its own state, which supports reproducibility (AG-2).

Table 8: Alternatives considered

Alternative	Reason not chosen
Web application (Flask or Django with a database)	No graphical interface or persistence is required; it would add deployment effort and attack surface without serving a requirement.
Microservices	A single-process batch computation gains nothing from network boundaries; latency, deployment and failure handling would increase substantially.
Discrete-event simulation library (for example SimPy)	Introduces a third-party dependency (violates C-1); all decisions occur at daily granularity, so a fixed time-step loop is simpler and easier to test deterministically.
Pipe-and-filter	Processing is not a linear flow; reorder decisions feed back into later days.

Table 9: Architecture decision records

ID	Decision	Rationale	Consequence
ADR-001	Layered architecture with a single orchestrator	Clear separation of concerns; parallel development; testability	Engine depends on every domain interface; it is integrated last
ADR-002	Fixed daily time step	Matches the granularity of all business rules; simple and deterministic	Intra-day timing (for example morning versus evening deliveries) is not modelled
ADR-003	Standard library only at runtime	Portability; no dependency failures in CI; minimal supply-chain risk	Statistics and CSV handling are implemented with standard modules
ADR-004	Scenario defined in a validated JSON file	Scenario changes need no code; validation fails fast on bad input	A schema must be maintained (Section 3.6)
ADR-005	One seeded random number generator injected into all stochastic components	Bit-for-bit reproducible runs	Components must never call the global random module
ADR-006	Unmet demand is backordered, not lost	Captures service failure and penalty cost explicitly	Lost-sales policies are a future enhancement (Section 4.6)

ID	Decision	Rationale	Consequence
ADR-007	One feature per branch and pull request, merged by squash	One commit per feature on main; clean history; balanced attribution	Intermediate branch commits are not preserved on main

3.6 Technology Stack & Data Stores

Table 10: Technology stack

Concern	Technology	Notes
Language and runtime	Python 3.12	CPython; executed inside a virtual environment
Runtime libraries	Python standard library	dataclasses, json, csv, math, random, statistics, argparse, pathlib
Testing	pytest 8.3.3	Pinned in requirements.txt; configured by pytest.ini
Continuous integration	GitHub Actions on ubuntu-latest	actions/checkout@v4, actions/setup-python@v5; job name test
Version control	Git with GitHub	Branch ruleset protect-main on the default branch
Design documentation	PlantUML	Source for all UML figures in this document

Data stores. The system uses no database. Three stores exist:

Table 11: Data stores

Store	Format	Direction	Lifetime
Scenario configuration	JSON file (default data/sample_config.json)	Input, read once at start	Persistent, version controlled
Daily results	CSV file (default output/results.csv)	Output, written once at end	Persistent, not version controlled
Simulation state	Python objects in memory	Internal	One run

Table 12: Configuration schema

Section	Key fields	Validation
simulation	days, seed	days is an integer from 1 to 3650
policy	supplier_strategy, service_z, forecast_method, forecast_window, smoothing_alpha, max_supplier_delay_days	Strategy and method from the allowed sets; window and delay at least 1; alpha in (0, 1]
warehouse	warehouse_id, name, initial_stock	Initial stock non-negative; every product exists in products
products	product_id, name, unit_cost, holding_cost_per_day, stockout_penalty, order_cost	Unique IDs; non-empty names; costs non-negative
suppliers	supplier_id, name, lead_time_days, prices, reliability, max_order_qty, distance_km, transport_mode	Lead time at least 1; reliability in [0, 1]; mode exists in transport_modes
demand	Per product: pattern, base, and spread or amplitude and period	Pattern in constant, uniform, seasonal; product exists
transport_modes	Per mode: cost_per_unit_km	Non-negative

Section	Key fields	Validation
Cross-section		Every product is sold by at least one supplier; file size at most 1 MB

Table 13: Daily results record (CSV columns, in order)

Column	Type	Description
day	int	Simulation day, starting at 0
product_id	str	Product identifier
demand	int	New customer demand on the day
shipped	int	Units shipped on the day, including fulfilment of earlier backorders
stock_end	int	Warehouse stock at end of day
backorder	int	Units owed to customers at end of day
on_order	int	Units on open purchase orders at end of day
forecast	float	Daily demand forecast made at end of day

Deployment and delivery view. The application runs locally inside a Python virtual environment on the user's workstation, reading the scenario file and writing results to the output folder (Figure 3). Source changes reach the shared repository only through feature branches and pull requests. Every pull request and every push to main triggers the CI workflow, which checks out the code, installs Python 3.12 and the pinned requirements, and runs the full pytest suite. The repository ruleset blocks merging until the status check test passes, at least one other team member approves, stale approvals are dismissed after new commits, conversations are resolved and the branch is up to date with main. Only squash merges are allowed, force pushes and branch deletion are blocked, and the bypass list is empty.

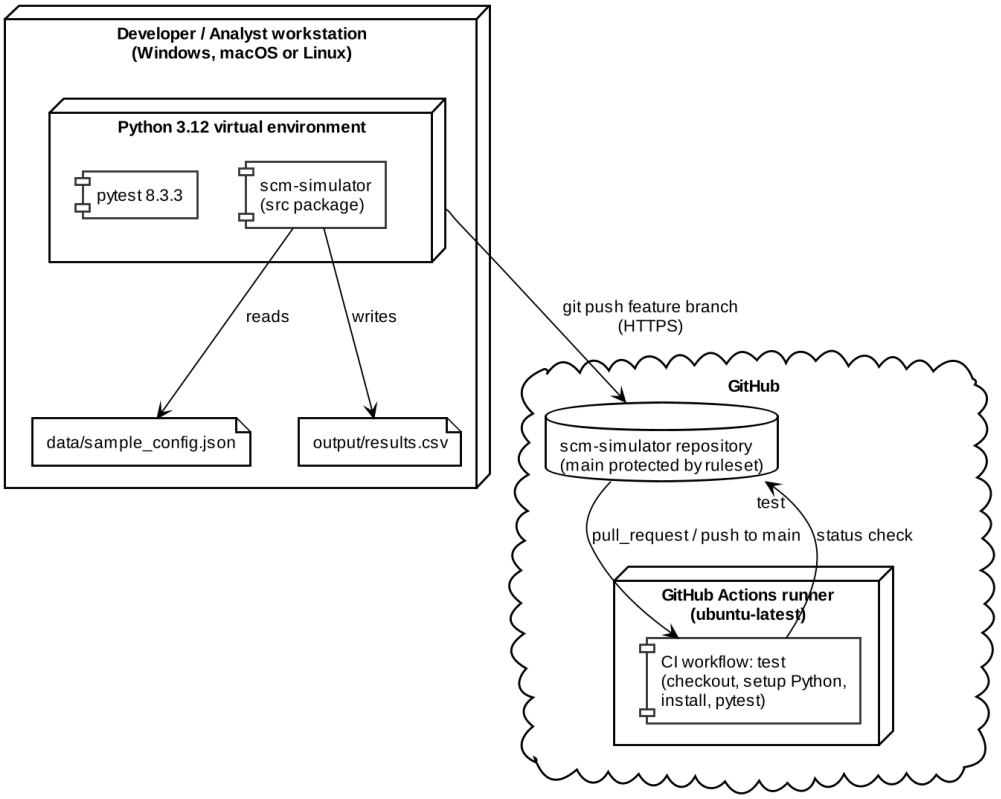


Figure 3: Deployment and delivery view

3.7 Risks & Mitigations

Table 14: Architecture risks

ID	Risk	Likelihood	Impact	Mitigation
R-1	Integration defects when the engine composes all modules late in development	Medium	High	Interfaces fixed in Section 4.3; engine merged last; integration tests TC-SIM-01 to 06
R-2	Merge conflicts between four developers	Medium	Medium	Single owner per module; shared files frozen after setup
R-3	Floating-point noise causes off-by-one results in rounded-up quantities	Medium	Medium	Round to 6 decimals before ceiling; known-value unit tests
R-4	Non-reproducible results from unseeded randomness	Low	High	Single injected RNG (ADR-005); TC-DEM-03, TC-PO-13, TC-SIM-03
R-5	Malformed or extreme configuration crashes or stalls the run	Medium	Medium	Schema and range validation, 1 MB size limit, 3650-day limit
R-6	Required CI check never reported, blocking pull requests	Low	Medium	No path filters in the workflow; fixed check name test
R-7	Simplified model reduces realism (single warehouse, no capacity limits)	High	Low	Scope agreed in the SRS; enhancements listed in Section 4.6

3.8 Traceability to Requirements

Table 15: Requirement to design element to test traceability

Requirement	Summary	Component (module)	Design element	Test cases
SCM-F-001	Product model and validation	products.py	Product	TC-PRD-01 to 04
SCM-F-002	Product catalog	products.py	ProductCatalog	TC-PRD-05 to 10
SCM-F-003	Warehouse stock receive and ship	inventory.py	Warehouse	TC-INV-01 to 06
SCM-F-004	Economic order quantity	inventory.py	economic_order_quantity	TC-INV-07 to 10
SCM-F-005	Safety stock, reorder point, reorder check	inventory.py	safety_stock, reorder_point, Warehouse.needs_reorder	TC-INV-11 to 15
SCM-F-006	Cost tracking	costs.py	CostTracker	TC-CST-01 to 04
SCM-F-010	Supplier model and validation	suppliers.py	Supplier	TC-SUP-01 to 05
SCM-F-011	Supplier registry	suppliers.py	SupplierRegistry	TC-SUP-06 to 09
SCM-F-012	Supplier selection strategies	suppliers.py	select_supplier	TC-SUP-10 to 15
SCM-F-013	Purchase order creation	purchase_orders.py	PurchaseOrder, create_purchase_order	TC-PO-01 to 05
SCM-F-014	PO lifecycle and order book	purchase_orders.py	PurchaseOrder.advance, PurchaseOrderBook	TC-PO-06 to 10
SCM-F-015	Supplier delivery delays	purchase_orders.py	apply_random_delay	TC-PO-11 to 14
SCM-F-020	Demand generation	demand.py	DemandGenerator	TC-DEM-01 to 05
SCM-F-021	Customer order model	orders.py	CustomerOrder	TC-ORD-06 to 09
SCM-F-022	Customer order book	orders.py	OrderBook	TC-ORD-10 to 13
SCM-F-023	FIFO allocation and backorders	orders.py	OrderBook.allocate, OrderBook.backorder_qty	TC-ORD-14 to 17
SCM-F-024	Moving average forecast and variability	forecasting.py	moving_average, demand_std	TC-FC-01 to 05
SCM-F-025	Exponential smoothing and error metrics	forecasting.py	exponential_smoothing, mean_absolute_error, mean_absolute_percentage_error	TC-FC-06 to 10
SCM-F-030	Simulation clock and event log	clock.py	SimulationClock, EventLog	TC-CLK-01 to 05
SCM-F-031	Configuration loading and validation	config_loader.py	load_config, validate_config	TC-CFG-01 to 06
SCM-F-032	Shipments and transport cost	shipments.py	Shipment, ShipmentLog	TC-SHP-01 to 04

Requirement	Summary	Component (module)	Design element	Test cases
SCM-F-033	KPI computation	kpis.py	compute_kpis and helpers	TC-KPI-01 to 05
SCM-F-034	CSV export and summary	reports.py	export_daily_csv, format_summary	TC-RPT-01 to 05
SCM-F-035	Simulation engine and CLI	simulation.py, main.py	SupplyChainSimulation, main	TC-SIM-01 to 06
SCM-NF-001	Performance	simulation.py	Linear daily loop, in-memory state	TC-NFR-01
SCM-NF-002	Reproducibility	simulation.py, demand.py, purchase_orders.py	Injected seeded RNG (ADR-005)	TC-NFR-02, TC-SIM-03
SCM-NF-003	Maintainability and test coverage	tests/, ci.yml	Unit tests per module, CI on every change	CI run history
SCM-NF-004	Usability of CLI and errors	main.py	argparse help, one-line errors, exit codes	TC-SIM-06
SCM-NF-005	Portability	All modules	Standard library only (ADR-003)	TC-NFR-03
SCM-SR-001	Configuration validation	config_loader.py	validate_config	TC-CFG-02 to 06
SCM-SR-002	Input size and run length limits	config_loader.py	MAX_CONFIG_BYTES, MAX_DAYS	TC-CFG-06
SCM-SR-003	CSV formula injection protection	reports.py	Cell sanitisation in export_daily_csv	TC-RPT-03, TC-RPT-04
SCM-SR-004	Protected main branch	Repository settings	Ruleset protect-main	Ruleset inspection; direct push rejected
SCM-SR-005	Pinned dependencies, no secrets committed	requirements.txt, .gitignore	Exact version pins; ignore rules	Pull request review

3.9 Security Architecture

The simulator processes no personal data, stores no credentials and exposes no network service at runtime, so its attack surface is small. The relevant assets are the integrity of the source code and delivery pipeline, the integrity of the scenario configuration, and the safety of the exported results for the person who opens them.

Table 16: Trust boundaries

ID	Boundary	Data crossing	Control point
TB-1	Configuration file to Config Loader	Untrusted JSON supplied by the user	load_config, validate_config
TB-2	CSV output to spreadsheet application	Simulation results containing configuration-derived strings	export_daily_csv
TB-3	Developer workstation to shared repository	Source code changes	Pull request, review, CI, branch ruleset

Table 17: Threat model (STRIDE)

Threat	Scenario	Boundary	Control	Requirement
Spoofing	A non-member pushes code or impersonates a team member	TB-3	Only invited collaborators can push; every change needs another member's approval	SCM-SR-004
Tampering	Configuration edited with invalid or inconsistent values; direct edits to main	TB-1, TB-3	Schema, range and reference validation; protected main with required CI	SCM-SR-001, SCM-SR-004
Repudiation	A change or simulation outcome cannot be attributed	TB-3	Git history, pull request reviews, CI logs; EventLog for each run	SCM-SR-004
Information disclosure	Secrets or local environment files committed	TB-3	.gitignore for environment files; review checklist	SCM-SR-005
Denial of service	Oversized file or excessive run length exhausts resources	TB-1	1 MB file limit; maximum 3650 days	SCM-SR-002
Elevation of privilege	A CSV cell executes as a spreadsheet formula; a member bypasses review	TB-2, TB-3	Prefix cells starting with =, +, - or @; empty ruleset bypass list	SCM-SR-003, SCM-SR-004

4. Design

4.1 Design Overview

Design principles.

- One responsibility per module; each module has a single owner.
- Dependencies point downward only; Domain modules never import the engine, the CLI or each other except where shown in Figure 1.
- The Simulation Engine is the only orchestrator; all business rules live in Domain modules.
- Input is validated at the boundary (fail fast); invalid values inside the Domain raise exceptions rather than being silently corrected.
- All randomness comes from one seeded RNG passed in by the engine; products are processed in ascending product_id order. Together these make runs reproducible.
- File and console I/O occur only in config_loader.py, reports.py and main.py.

Processing model. Each simulated day the engine performs the following steps:

1. Receive purchase orders due on or before the day: add stock, mark the order DELIVERED, record transport cost and log PO_RECEIVED.
2. For each product in catalog order: generate demand and create a customer order (Figure 7).
3. Allocate available stock to open customer orders oldest first, ship the allocated units, and record holding and stockout cost (Figure 7).
4. Forecast demand, select a supplier, compute safety stock and reorder point, and if stock plus open orders is at or below the reorder point, place a purchase order sized by EOQ and apply a possible delivery delay (Figure 6).

5. Append the daily record for the product, then advance the clock.

After the final day, the engine computes KPIs from the daily records, adds the forecast error, and returns the cost breakdown, KPIs, records and event log as a SimulationResult.

Table 18: Core formulas

Quantity	Formula	Component
Economic order quantity	EOQ = ceil(sqrt(2 x D x S / H)); D = annual demand, S = order cost, H = annual holding cost per unit	inventory.py
Safety stock	SS = ceil(z x sigma_d x sqrt(L)); sigma_d = daily demand standard deviation, L = lead time in days	inventory.py
Reorder point	ROP = ceil(d x L + SS); d = forecast daily demand	inventory.py
Reorder condition	stock + on_order <= ROP	inventory.py
Balanced supplier score	0.5 x (price / max price) + 0.3 x (lead time / max lead time) + 0.2 x (1 - reliability); lowest wins	suppliers.py
Exponential smoothing	F(t+1) = alpha x A(t) + (1 - alpha) x F(t), with F(1) = A(1)	forecasting.py
Holding / stockout cost per day	stock_end x holding_cost_per_day; backorder x stockout_penalty	costs.py
Transport cost	distance_km x quantity x cost_per_unit_km(mode)	shipments.py
Fill rate	total shipped / total demand	kpis.py
Service level	1 - (product-days with backorder > 0) / (product-days)	kpis.py
Inventory turnover	COGS / average inventory value	kpis.py

Object lifecycles. Figure 4 shows the permitted state transitions for purchase orders and customer orders. Any transition not shown raises an error.

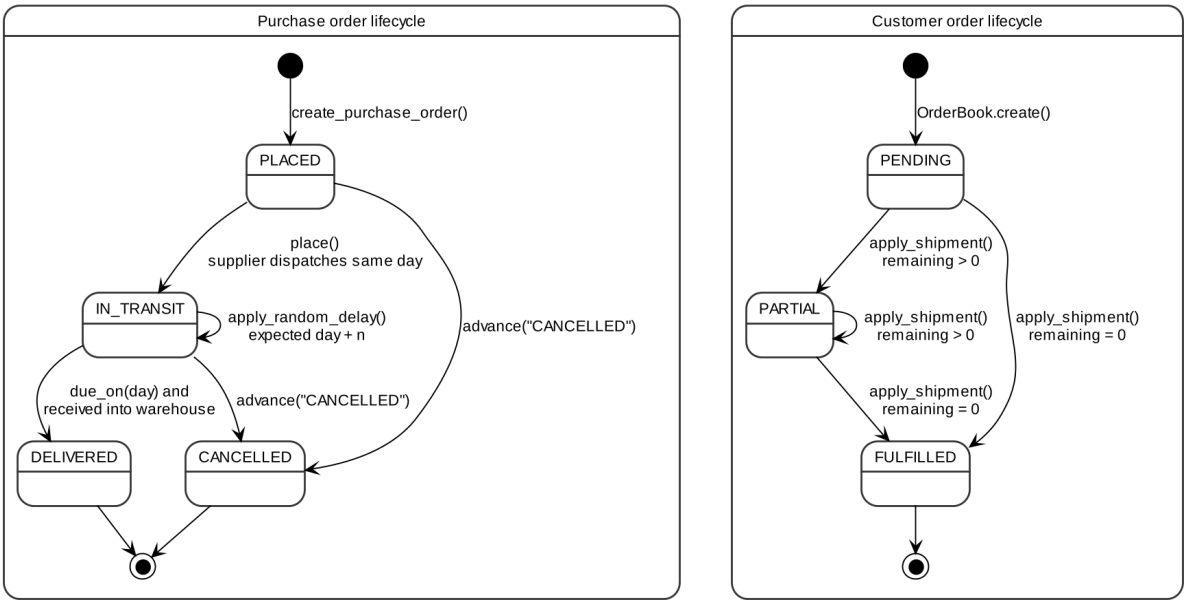


Figure 4: State diagrams for purchase orders and customer orders

4.2 UML Sequence Diagrams

Three sequence diagrams describe the main runtime flows. Figure 5 shows the end-to-end run, including a complete simulated day; it references Figures 6 and 7 for the per-product

steps. Figure 6 shows the replenishment decision and purchase order placement. Figure 7 shows demand capture and order fulfilment with backorders.

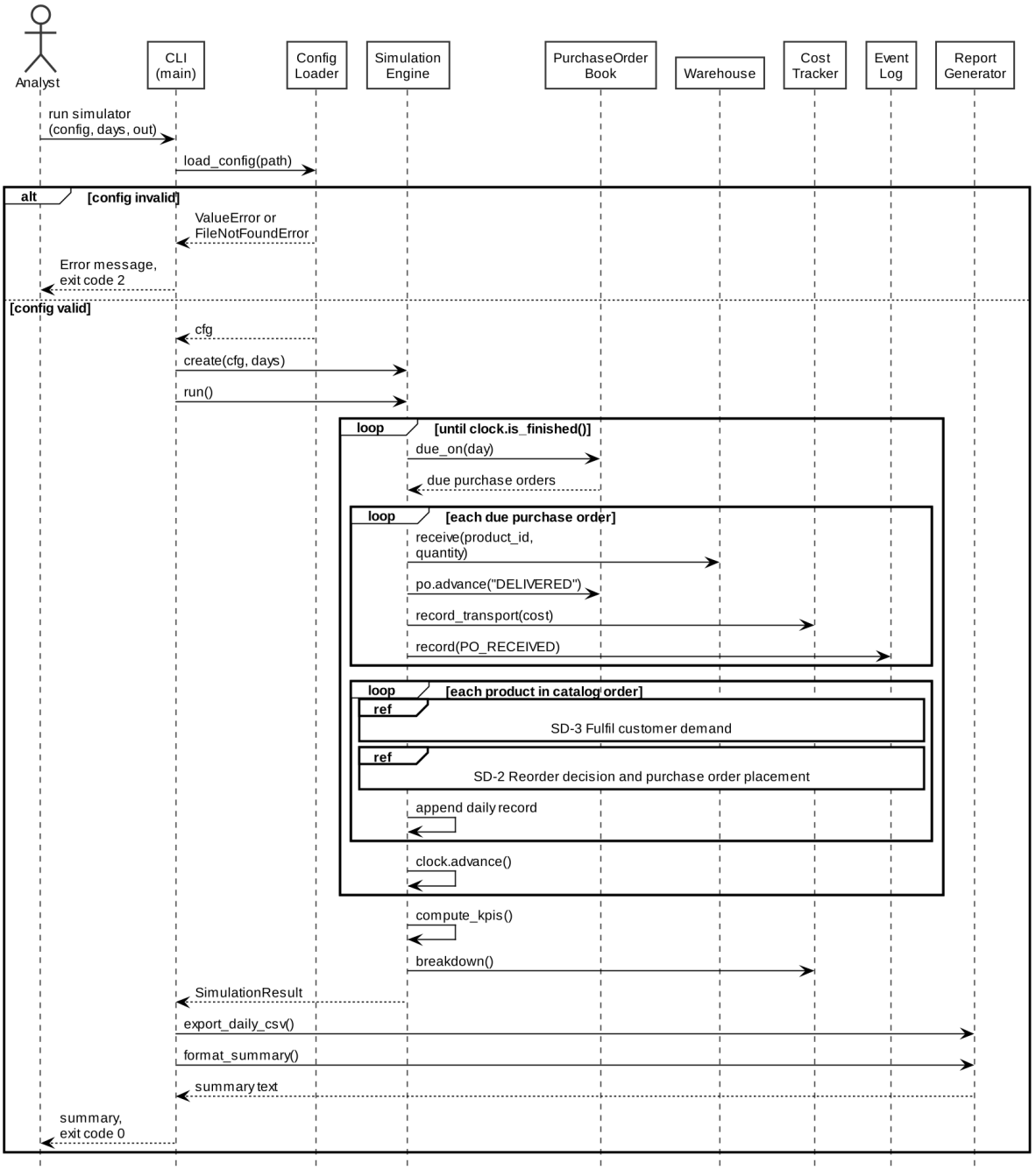


Figure 5: SD-1 Run simulation and process one day

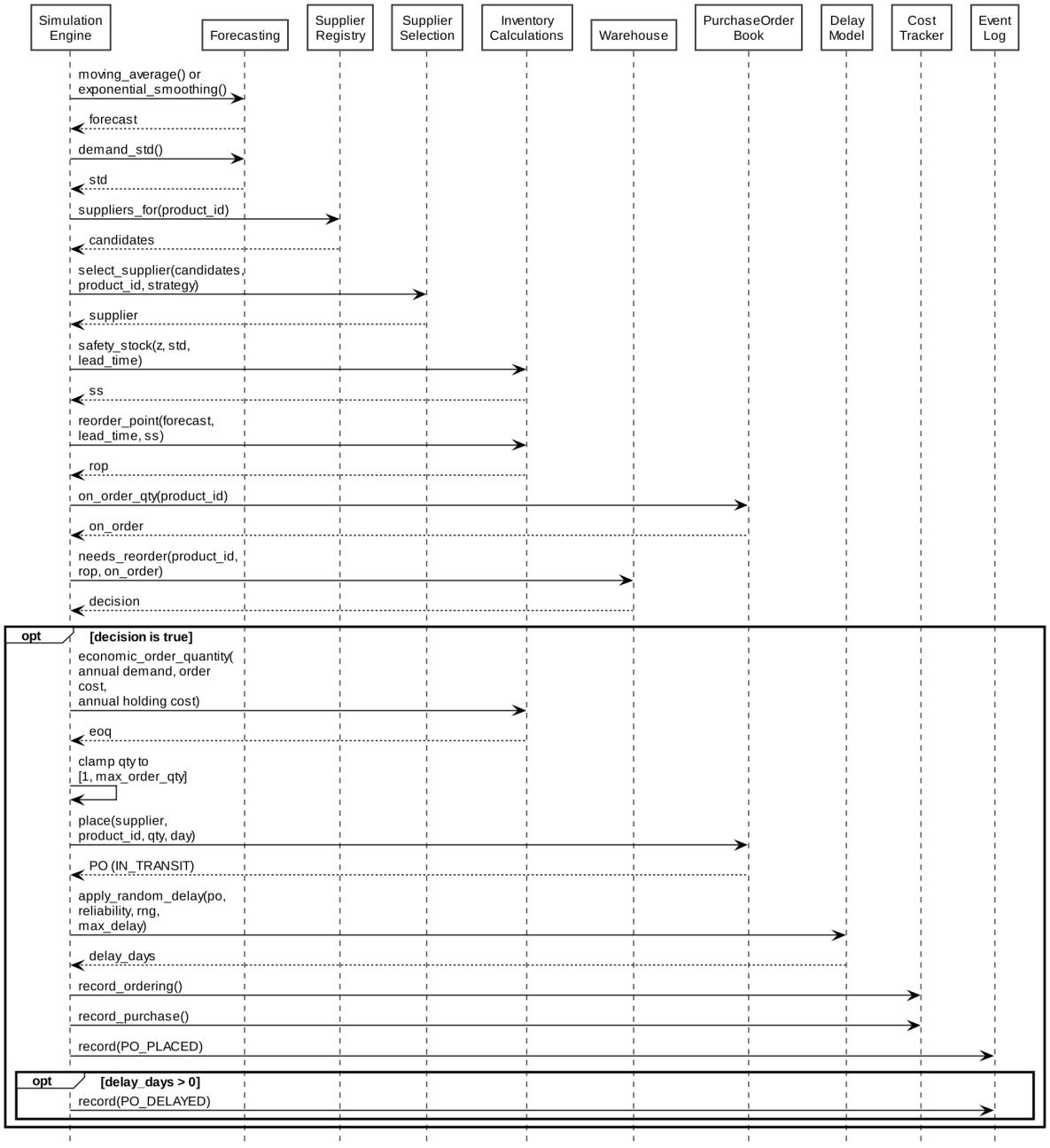


Figure 6: SD-2 Reorder decision and purchase order placement

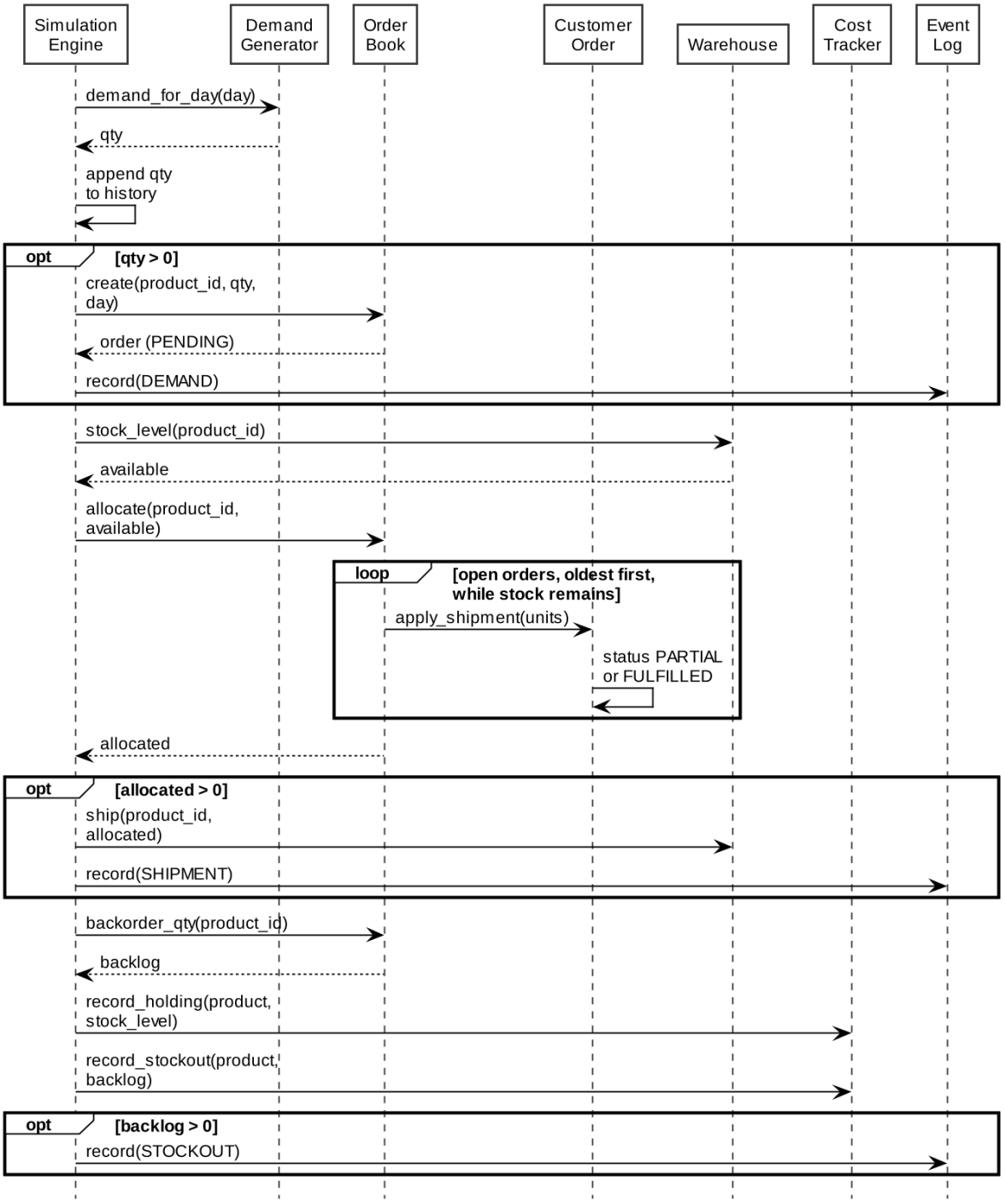


Figure 7: SD-3 Fulfil customer demand with backorders

4.3 API Design

The simulator exposes one external interface (the command line) and a set of internal component interfaces. Internal interfaces are Python callables; their contracts below are binding for all modules. Errors are raised as Python exceptions.

4.3.1 Command-line interface (external)

Table 19: CLI contract

Item	Definition
Command	python -m src.main [--config PATH] [--days N] [--out PATH]
--config	Scenario file. Default: data/sample_config.json
--days	Overrides simulation.days. Integer from 1 to 3650
--out	CSV output path; parent folders are created. Default: output/results.csv
--help	Prints usage and exits with code 0
Response	Summary printed to standard output; CSV written to --out
Exit codes	0 success; 2 invalid input (missing or invalid configuration, invalid arguments); 1 unexpected internal error
Error output	One line on standard error, prefixed with Error:, no stack trace

4.3.2 Inventory component (inventory.py)

Table 20: Inventory interface

Operation	Input	Output	Errors
Warehouse(warehouse_id, name, initial_stock)	IDs as strings; initial_stock as product_id to units	Warehouse instance	ValueError: negative initial stock
Warehouse.receive(product_id, qty)	qty: positive int	None; stock increases by qty	ValueError: qty not positive
Warehouse.ship(product_id, qty)	qty: positive int	int: units shipped = min(qty, stock)	ValueError: qty not positive
Warehouse.stock_level(product_id)	product_id	int: current stock, 0 if never stocked	None
Warehouse.needs_reorder(product_id, reorder_point, on_order)	Non-negative ints	bool: stock + on_order <= reorder_point	ValueError: negative input
economic_order_quantity(annual_demand, order_cost, annual_holding_cost_per_unit)	Non-negative floats; holding cost > 0	int: EOQ rounded up; 0 if demand is 0	ValueError: holding cost <= 0 or negative input
safety_stock(z, demand_std_per_day, lead_time_days)	Non-negative numbers	int: safety stock rounded up	ValueError: negative input
reorder_point(avg_daily_demand, lead_time_days, safety_stock_units)	Non-negative numbers	int: reorder point rounded up	ValueError: negative input

4.3.3 Procurement component (suppliers.py, purchase_orders.py)

Table 21: Procurement interface

Operation	Input	Output	Errors
SupplierRegistry.suppliers_for(product_id)	product_id	list[Supplier] sorted by supplier_id; empty if none	None

Operation	Input	Output	Errors
select_supplier(candidates, product_id, strategy, weights)	strategy: cheapest, fastest or balanced	Supplier with the best value; ties to lower supplier_id	ValueError: no supplier for product or unknown strategy
PurchaseOrderBook.place(supplier, product_id, quantity, order_day)	1 <= quantity <= supplier.max_order_qty	PurchaseOrder with next ID (PO-0001, ...), status IN_TRANSIT, expected_delivery_day = order_day + lead time	ValueError: quantity out of range or product not supplied
PurchaseOrderBook.due_on(day)	day: int	Open orders with expected_delivery_day <= day, sorted by po_id	None
PurchaseOrderBook.on_order_qty(product_id)	product_id	int: units on PLACED and IN_TRANSIT orders	None
PurchaseOrder.advance(new_status)	Target status	None; status updated	ValueError: transition not permitted (Figure 4)
apply_random_delay(order, reliability, rng, max_delay_days)	0 <= reliability <= 1; max_delay_days >= 1	int: delay in days (0 if on time); expected_delivery_day updated	ValueError: invalid input or closed order

4.3.4 Order management component (orders.py)

Table 22: Order management interface

Operation	Input	Output	Errors
OrderBook.create(product_id, quantity, day)	quantity >= 1	CustomerOrder with next ID (CO-0001, ...), status PENDING	ValueError: quantity < 1
OrderBook.open_orders(product_id)	product_id	PENDING and PARTIAL orders, oldest first	None
OrderBook.allocate(product_id, available_qty)	available_qty >= 0	int: units allocated, oldest orders first	ValueError: negative quantity
OrderBook.backorder_qty(product_id)	product_id	int: units still owed on open orders	None
CustomerOrder.apply_shipment(qty)	1 <= qty <= remaining	None; status PARTIAL or FULFILLED	ValueError: qty out of range

4.3.5 Configuration component (config_loader.py)

Table 23: Configuration interface

Operation	Input	Output	Errors
load_config(path)	Path to a JSON file of at most 1 MB	dict: validated configuration	FileNotFoundError: missing file; ValueError: oversized file, invalid JSON or failed validation

Operation	Input	Output	Errors
validate_config(cfg)	dict	None	ValueError naming the first failing section or field

4.4 Error Handling, Logging & Monitoring

Error handling policy. Components validate their inputs and raise standard exceptions; they do not print, log or exit. The CLI is the single place where exceptions are converted into user-facing messages and exit codes.

Table 24: Exception handling

Exception	Raised when	Raised by	Handling in the CLI
FileNotFoundError	Configuration file does not exist	config_loader.py	Print Error: message; exit code 2
ValueError	Invalid configuration, argument or domain value; invalid state transition	All components	Print Error: message; exit code 2
KeyError	Unknown product, supplier or order identifier	Catalogues and order books	Indicates a defect when raised during a run; exit code 1
RuntimeError	Clock advanced beyond the final day	clock.py	Indicates a defect; exit code 1
Any other exception	Unexpected failure	Any	Print Error: unexpected failure with the exception message; exit code 1

Logging. Each run records structured events in an in-memory EventLog, returned in the SimulationResult. Domain modules do not write to the console. No sensitive data exists in the system, so no masking is required.

Table 25: Event types

Event type	Emitted when	Message content
DEMAND	A customer order is created	Product, quantity, order ID
SHIPMENT	Units are shipped to customers	Product, units shipped
STOCKOUT	Backorder remains at end of day	Product, backorder units
PO_PLACED	A purchase order is placed	PO ID, supplier, product, quantity, expected delivery day
PO_DELAYED	A delay is applied to a new purchase order	PO ID, delay in days, new expected delivery day
PO_RECEIVED	A purchase order is delivered	PO ID, product, quantity, transport cost

Monitoring. Code health is monitored by the CI workflow on every pull request and every push to main; the GitHub Actions history provides the audit trail. A failing build on main suspends further merges until it is fixed. Run-level health is reported through the KPIs (fill rate, service level, inventory turnover, forecast error) and the event counts.

4.5 UX Design

The user interacts through a terminal. Design choices:

- A single command with sensible defaults; the simulator runs with no options at all.
- Plain text output that does not rely on colour, so it works in any terminal and with screen readers.
- Units shown explicitly: costs as Rs with two decimals, rates as percentages with two decimals.
- Errors on one line prefixed with Error:, naming the field or file at fault, with no stack trace.
- Results also exported to CSV for analysis and charting in a spreadsheet.

Table 26: Console summary layout

Block	Content
Run	Configuration file, number of days, seed, forecast method, supplier strategy
Costs	Holding, stockout, ordering, purchase, transport and total cost (Rs, 2 decimals)
Service	Fill rate and service level (percent, 2 decimals); backorder units at end of run
Inventory	Average inventory units and inventory turnover
Forecast	Mean absolute error of the daily forecast
Activity	Counts of purchase orders placed, delayed and received; stockout product-days
Output	Path of the CSV file written

4.6 Open Issues & Next Steps

Table 27: Open issues and planned enhancements

ID	Item	Type	Next step
OI-1	Only one warehouse is modelled	Limitation	Introduce a network of warehouses with transfer orders
OI-2	Unmet demand is always backordered (ADR-006)	Limitation	Add a configurable lost-sales policy
OI-3	Supplier capacity is limited per order, not per period	Limitation	Add per-period supplier capacity
OI-4	A single run gives one sample path of a stochastic process	Enhancement	Monte Carlo mode: repeat runs with different seeds and report confidence intervals
OI-5	Results are viewed as text and CSV only	Enhancement	Optional dashboard over the CSV output
OI-6	Policy comparison requires separate runs	Enhancement	Batch mode that compares strategies and forecasting methods side by side

5. Appendices

5.1 Glossary

Table 28: Glossary

Term	Meaning
Backorder	Customer demand accepted but not yet shipped because of insufficient stock
Fill rate	Share of demanded units that were shipped
Holding cost	Cost of keeping one unit in stock for one day
Lead time	Days between placing a purchase order and its expected delivery
Reorder point	Stock level at or below which a replenishment order is placed
Safety stock	Extra stock held to absorb variation in demand during the lead time
Service level	Share of product-days without outstanding backorders
Stockout penalty	Cost charged per backordered unit per day
Inventory turnover	Cost of goods sold divided by average inventory value
Reliability	Probability that a supplier delivers on the expected day
Ruleset	GitHub configuration that enforces conditions before changes reach a branch

5.2 References

1. ISO/IEC/IEEE 42010:2022, Software, systems and enterprise: Architecture description. https://www.iso.org/standard/74393.html
2. IEEE 1016-2009, IEEE Standard for Information Technology: Systems Design, Software Design Descriptions. https://standards.ieee.org/ieee/1016/4502/
3. NIST SP 800-160 Vol. 1 Rev. 1, Engineering Trustworthy Secure Systems. https://csrc.nist.gov/pubs/sp/800/160/v1/r1/final
4. Microsoft, Threat Modeling Tool threats (STRIDE categories). https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats
5. OWASP, CSV Injection. https://owasp.org/www-community/attacks/CSV_Injection
6. Harris, F. W., How Many Parts to Make at Once, reprinted in Operations Research 38(6):947 to 950, 1990. https://doi.org/10.1287/opre.38.6.947
7. Hyndman, R. J. and Athanasopoulos, G., Forecasting: Principles and Practice, 3rd edition. https://otexts.com/fpp3/
8. GitHub Docs, About rulesets. https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/about-rulesets
9. GitHub Docs, GitHub Actions documentation. https://docs.github.com/en/actions
10. Python Software Foundation, The Python Standard Library (3.12). https://docs.python.org/3.12/library/

5.3 Tools

Table 29: Tools

Tool	Use
PlantUML	Component, class, deployment, state and sequence diagrams

Tool	Use
Visual Studio Code	Development environment
Git and GitHub	Version control, pull requests, code review, branch rulesets
GitHub Actions	Continuous integration
pytest	Unit and integration testing
Python venv	Isolated runtime environment